// Security Assessment

06.23.2026 - 06.26.2026

Remove Openbook Dependency From AmmV4

Raydium

HALBORN

Remove Openbook Dependency From AmmV4 - Raydium

Prepared by: HALBORN

Last Updated 07/08/2026

Date of Engagement: June 23rd, 2026 - June 26th, 2026

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
8	0	0	0	1	7

TABLE OF CONTENTS

1. Introduction
2. Assessment summary
3. Test approach and methodology
4. Automated testing
5. Static analysis report
   5.1 Description
6. Risk methodology
7. Scope
8. Assessment summary & findings overview
9. Findings & Tech Details
   9.1 In-place upgrade could brick legacy pools and trap liquidity without on-chain recovery path
   9.2 Unreachable macro may cause panic on out-of-range instruction data
   9.3 Unchecked 32-byte read in updateconfigaccount panics on truncated instruction data instead of returning a controlled error
   9.4 Stale openbook accounts in builders misalign the signer slot, breaking withdrawpnl and set params
   9.5 Mutable amm owner field with no effect on access control
   9.6 Residual openbook validation with no effect during setting params

9.7 Statistical fields in statedata remain permanently at zero
9.8 inconsistent comparison in params update prevents reducing max price multiplier

1. Introduction

The `Raydium team` engaged `Halborn` to conduct a security assessment of a set of changes in their **Radium AMM Solana program** beginning on June 23, 2026, and ending on June 26, 2026. The security assessment was scoped to the Solana Program provided in raydium-amm GitHub repository. Commit hashes and further details can be found in the Scope section of this report.

The changes implemented in the `raydium-amm` program remove the OpenBook/Serum (`serum_dex`) dependency, turning the AMM into a standalone constant-product pool that no longer relies on an on-chain order book. All order-book logic is deleted and reserves are now computed solely from vault balances. As a result, the order-book-dependent instructions (MonitorStep, MigrateToOpenBook, WithdrawSrm, SimulateInfo, AdminCancelOrders) are retired and now return `unimplemented!()`. The entrypoint's error handling is rewritten with explicit `msg!` logging, the `solana-program` pin is relaxed, and a LiteSVM-based integration-test harness is added.

2. Assessment Summary

`Halborn` was provided 4 days for the engagement and assigned one full-time security engineer to review the security of the Solana Programs in scope. The engineer is a blockchain and smart contract security expert with advanced smart contract hacking skills, and deep knowledge of multiple blockchain protocols.

The purpose of the assessment is to:

- Identify potential security issues within the Solana Programs.
- Ensure that smart contract functionality operates as intended.

In summary, `Halborn` identified some improvements to reduce the likelihood and impact of risks, most of which have been addressed by `Raydium` team. The primary findings were as follows:

- `Verify on-chain whether any pools have status == 1/5 or non-zero open_orders balance and consider reordering process_set_params to allow AmmParams::Status changes even when orderbook_permission() is true.`
- `Refactor AmmParams::from_u64 and SimulateParams::from_u64 to return a Result type instead of invoking a panic.`
- `Apply the same length guard used in tag 6 to the tag 15 deserialization path.`
- `Realign each builder with its processor by removing accounts the processor no longer consumes, ensuring positional order matches what the processor reads.`
- `Remove the AmmParams::AmmOwner branch from process_set_params.`
- `Remove the residual validation and discard the account binding.`
- `Explicitly document that these StateData fields are deprecated and will remain at zero.`
- `Align the comparison with the MinPriceMultiplier case to allow bidirectional updates.`

3. Test Approach And Methodology

Halborn performed a combination of a manual review of the source code and automated security testing to balance efficiency, timeliness, practicality, and accuracy in regard to the scope of the program assessment. While manual testing is recommended to uncover flaws in business logic, processes, and implementation; automated testing techniques help enhance coverage of programs and can quickly identify items that do not follow security best practices.

The following phases and associated tools were used throughout the term of the assessment:

- Research into the architecture, purpose, and use of the platform.
- Manual program source code review to identify business logic issues.
- Mapping out possible attack vectors
- Thorough assessment of safety and usage of critical Rust variables and functions in scope that could lead to arithmetic vulnerabilities.
- Scanning dependencies for known vulnerabilities (`cargo audit`).
- Local runtime testing (`cargo test `)

4. Automated Testing

5. Static Analysis Report

5.1 Description

Halborn used automated security scanners to assist with detection of well-known security issues and vulnerabilities. Among the tools used was `cargo audit`, a security scanner for vulnerabilities reported to the RustSec Advisory Database. All vulnerabilities published in `https://crates.io` are stored in a repository named The RustSec Advisory Database. `cargo audit` is a human-readable version of the advisory database which performs a scanning on Cargo.lock. Security Detections are only in scope. All vulnerabilities shown here were already disclosed in the above report. However, to better assist the developers maintaining this code, the auditors are including the output with the dependencies tree, and this is included in the cargo audit output to better know the dependencies affected by unmaintained and vulnerable crates.

Cargo Audit Results

ID	CRATE	DESCRIPTION
RUSTSEC-2024-0344	curve25519-dalek	Timing variability in `curve25519-dalek`'s `Scalar29::sub`/`Scalar52::sub`
RUSTSEC-2022-0093	ed25519-dalek	Double Public Key Signing Function Oracle Attack on `ed25519-dalek`
RUSTSEC-2026-0099	rustls-webpki	Name constraints were accepted for certificates asserting a wildcard name
RUSTSEC-2026-0098	rustls-webpki	Name constraints for URI names were incorrectly accepte
RUSTSEC-2026-0104	rustls-webpki	Reachable panic in certificate revocation list parsing

6. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

6.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

6.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N) Low (C:L) Medium (C:M) High (C:H) Critical (C:C)	0 0.25 0.5 0.75 1
Integrity (I)	None (I:N) Low (I:L) Medium (I:M) High (I:H) Critical (I:C)	0 0.25 0.5 0.75 1
Availability (A)	None (A:N) Low (A:L) Medium (A:M) High (A:H) Critical (A:C)	0 0.25 0.5 0.75 1
Deposit (D)	None (D:N) Low (D:L) Medium (D:M) High (D:H) Critical (D:C)	0 0.25 0.5 0.75 1
Yield (Y)	None (Y:N) Low (Y:L) Medium (Y:M) High (Y:H) Critical (Y:C)	0 0.25 0.5 0.75 1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

6.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N) Partial (R:P) Full (R:F)	1 0.5 0.25
Scope (s)	Changed (S:C) Unchanged (S:U)	1.25 1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4

SEVERITY	SCORE VALUE RANGE
Informational	0 - 1.9

7. SCOPE

REPOSITORY

(a) Repository: raydium-amm

(b) Assessed Commit ID: e7fb5b1

(c) Items in scope:

- program/Cargo.toml
- program/src/entrypoint.rs
- program/src/error.rs
- program/src/invokers.rs
- program/src/log.rs
- program/src/math.rs
- program/src/processor.rs
- program/src/state.rs

FILE

(a) Submitted File: raydium-amm-remove_openbook_dependency.zip

(b) Items in scope:

- /raydium-amm-remove_openbook_dependency/program/src/entrypoint.rs
- /raydium-amm-remove_openbook_dependency/program/src/error.rs
- /raydium-amm-remove_openbook_dependency/program/src/instruction.rs
- /raydium-amm-remove_openbook_dependency/program/src/invokers.rs
- /raydium-amm-remove_openbook_dependency/program/src/log.rs
- /raydium-amm-remove_openbook_dependency/program/src/lib.rs
- /raydium-amm-remove_openbook_dependency/program/src/math.rs
- /raydium-amm-remove_openbook_dependency/program/src/processor.rs
- /raydium-amm-remove_openbook_dependency/program/src/state.rs

REMEDIATION COMMIT ID:

- 02f3513
- b20ba56

Out-of-Scope: New features/implementations after the remediation commit IDs.

8. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
0	0	0	1	7

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
IN-PLACE UPGRADE COULD BRICK LEGACY POOLS AND TRAP LIQUIDITY WITHOUT ON-CHAIN RECOVERY PATH	LOW	PARTIALLY SOLVED - 07/02/2026
UNREACHABLE MACRO MAY CAUSE PANIC ON OUT-OF-RANGE INSTRUCTION DATA	INFORMATIONAL	SOLVED - 07/07/2026
UNCHECKED 32-BYTE READ IN UPDATECONFIGACCOUNT PANICS ON TRUNCATED INSTRUCTION DATA INSTEAD OF RETURNING A CONTROLLED ERROR	INFORMATIONAL	SOLVED - 07/02/2026
STALE OPENBOOK ACCOUNTS IN BUILDERS MISALIGN THE SIGNER SLOT, BREAKING WITHDRAWPNL AND SET PARAMS	INFORMATIONAL	SOLVED - 07/02/2026
MUTABLE AMM OWNER FIELD WITH NO EFFECT ON ACCESS CONTROL	INFORMATIONAL	SOLVED - 07/02/2026
RESIDUAL OPENBOOK VALIDATION WITH NO EFFECT DURING SETTING PARAMS	INFORMATIONAL	SOLVED - 07/02/2026
STATISTICAL FIELDS IN STATEDATA REMAIN PERMANENTLY AT ZERO	INFORMATIONAL	SOLVED - 07/02/2026

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
INCONSISTENT COMPARISON IN PARAMS UPDATE PREVENTS REDUCING MAX PRICE MULTIPLIER	INFORMATIONAL	SOLVED - 07/02/2026

9. FINDINGS & TECH DETAILS

9.1 IN-PLACE UPGRADE COULD BRICK LEGACY POOLS AND TRAP LIQUIDITY WITHOUT ON-CHAIN RECOVERY PATH

// LOW

Description

The migration removing OpenBook dependency converts orderbook lifecycle instructions into `unimplemented!()` stubs, calculates reserves from vaults-only, and introduces a guard across 8 handlers that rejects pools with `orderbook_permission() == true`:

program/src/processor.rs

Copy Code

```
1014 | if AmmStatus::from_u64(amm.status).orderbook_permission() {
1015 |     return Err(AmmError::InvalidStatus.into())
1016 | }
```

This guard appears in `deposit`, `withdrawpnl`, `withdraw`, all 4 swaps, and `set_params`. The function returns true only for `Initialized(1)` and `OrderBookOnly(5)` states. For fresh deployments with a new program ID, this would be harmless since new pools are born in `SwapOnly(6)` or `WaitingTrade(7)` with `open_orders = Pubkey::default()`. However, an **in-place upgrade** over an existing program ID could inherit legacy pools and create two potential issues.

By removing settle/cancel CPIs to OpenBook, the program would no longer be able to move balances out of `open_orders` accounts (owned by `amm_authority` PDA).

- **Aspect A**: If any legacy pool—even one in `SwapOnly(6)`—had liquidity in its `open_orders`, those funds could become orphaned. Reserves would be calculated only from vaults, so LPs would withdraw against understated balances while a portion remains permanently inaccessible.
- **Aspect B**: Pools in states 1 or 5 would become completely frozen—the guard would block all operations including `set_params`, which executes the guard check before the status write, creating an ordering deadlock with no on-chain recovery path.

program/src/state.rs

Copy Code

```
310 | pub fn orderbook_permission(&self) -> bool {
311 |   match self {
312 |       AmmStatus::Uninitialized => false,
313 |       AmmStatus::Initialized => true,
314 |       AmmStatus::Disabled => false,
315 |       AmmStatus::WithdrawOnly => false,
316 |       AmmStatus::LiquidityOnly => false,
317 |       AmmStatus::OrderBookOnly => true,
318 |       AmmStatus::SwapOnly => false,
319 |
```

```
        AmmStatus::WaitingTrade => false,
    }
```

program/src/math.rs

Copy Code

```rust
pub fn calc_total_without_take_pnl_no_orderbook<'a>(
    pc_amount: u64,
    coin_amount: u64,
    amm: &'a AmmInfo,
) -> Result<(u64, u64), AmmError> {
    let total_pc_without_take_pnl = pc_amount
        .checked_sub(amm.state_data.need_take_pnl_pc)
        .ok_or(AmmError::CheckedSubOverflow)?;
    let total_coin_without_take_pnl = coin_amount
        .checked_sub(amm.state_data.need_take_pnl_coin)
        .ok_or(AmmError::CheckedSubOverflow)?;
    Ok((total_pc_without_take_pnl, total_coin_without_take_pnl))
```

program/src/processor.rs

Copy Code

```rust
let mut amm = AmmInfo::load_mut_checked(&amm_info, program_id)?;
if AmmStatus::from_u64(amm.status).orderbook_permission() {
    return Err(AmmError::InvalidStatus.into());
}
if *amm_authority_info.key
    != Self::authority_id(program_id, AUTHORITY_AMM, amm.nonce as u8)?
{
    return Err(AmmError::InvalidProgramAddress.into());
}
if amm_info.owner != program_id {
    return Err(AmmError::InvalidOwner.into());
}
if !amm_owner_info.is_signer || *amm_owner_info.key != config_feature::amm_owner::ID {
    return Err(AmmError::InvalidSignAccount.into());
}
if *amm_open_orders_info.key != amm.open_orders {
    return Err(AmmError::InvalidOpenOrders.into());
}
if amm.coin_vault != *amm_coin_vault_info.key {
    return Err(AmmError::InvalidCoinVault.into());
}
if amm.pc_vault != *amm_pc_vault_info.key {
    return Err(AmmError::InvalidPCVault.into());
}
if amm.target_orders != *amm_target_orders_info.key {
    return Err(AmmError::InvalidTargetOrders.into());
}

let param = setparams.param;
let mut set_valid = false;
match AmmParams::from_u64(param as u64) {
    AmmParams::Status => {
        let value = match setparams.value {
            Some(a) => a,
            None => return Err(AmmError::InvalidInput.into()),
        };
        if AmmStatus::valid_status(value) {
```

BVSS

AO:A/AC:L/AX:M/R:P/S:U/C:N/A:H/I:M/D:N/Y:N (2.9)

Recommendation

Consider the following measures if an in-place upgrade is planned:

- Verify on-chain whether any pools have `status == 1/5` or non-zero `open_orders` balances. If found, migrate/settle those balances using the prior program version before upgrading, or retain a one-shot drain instruction for `amm_authority` to move funds from OpenOrders to vaults.
- Reordering `process_set_params` to allow `AmmParams::Status` changes even when `orderbook_permission() == true`, restricting the destination to safe states (not 1 or 5), providing a recovery path.

Remediation Comment

PARTIALLY SOLVED: The `Raydium` partially solved this issue by removing the `orderbook_permission()` guard from all 8 handlers — including `process_set_params`.
Nevertheless, the team stated that an in-place upgrade could brick legacy pools and trap liquidity without an on-chain recovery path. The team further stated that they reviewed all pools on mainnet and confirmed that all Raydium liquidity previously shared on OpenBook has been migrated to the AMM vaults. Based on this, the team concluded that direct upgrades should not currently present any issues.

Remediation Hash

02f351333e54b7fca573aad2f2b657ed6fe27903

9.2 UNREACHABLE MACRO MAY CAUSE PANIC ON OUT-OF-RANGE INSTRUCTION DATA

// INFORMATIONAL

Description

When deserializing instruction tags 6 (SetParams) and 12 (SimulateInfo), `AmmInstruction::unpack()` reads a `param` byte from user-supplied instruction data and passes it directly to `AmmParams::from_u64(param as u64)` and `SimulateParams::from_u64(param as u64)` respectively, without prior range validation.

Both conversion functions terminate their match statement with `_ => unreachable!()`: `AmmParams::from_u64` covers only values 0–17, and `SimulateParams::from_u64` covers only 0–3. Since `param` is a user-controlled `u8` (range 0–255), any value outside the defined enum range triggers `unreachable!()`, which produces a panic in the SBF runtime and aborts the transaction.

The issue arises because `unreachable!()` is intended to signal logically impossible code paths to the compiler and optimizer, not to validate external input. Using it for user-supplied discriminants treats invalid data as a programmer-level assertion failure rather than as malformed input that should be rejected with a controlled error. The panic occurs during deserialization, before any authority checks or state modifications, but the use of `unreachable!()` misrepresents the nature of the failure: the code path is not impossible, it is simply invalid user input that was not validated upstream.

Although this does not represent an exploitable security vulnerability—the impact is limited to aborting the transaction that the sender constructs and signs—it is recommended to replace the assertion-based control flow with explicit error handling.

program/src/instructions

Copy Code

```
465 | 6 => {
466 |   let (param, rest) = Self::unpack_u8(rest)?;
467 |   match AmmParams::from_u64(param as u64) {
```

program/src/instructions.rs

Copy Code

```
556 | 12 => {
557 |   let (param, rest) = Self::unpack_u8(rest)?;
558 |   match SimulateParams::from_u64(param as u64) {
```

program/src/state.rs

```rust
impl AmmParams {
  pub fn from_u64(state: u64) -> Self {
    match state {
        0u64 => AmmParams::Status,
        1u64 => AmmParams::State,
        2u64 => AmmParams::OrderNum,
        3u64 => AmmParams::Depth,
        4u64 => AmmParams::AmountWave,
        5u64 => AmmParams::MinPriceMultiplier,
        6u64 => AmmParams::MaxPriceMultiplier,
        7u64 => AmmParams::MinSize,
        8u64 => AmmParams::VolMaxCutRatio,
        9u64 => AmmParams::Fees,
        10u64 => AmmParams::AmmOwner,
        11u64 => AmmParams::SetOpenTime,
        12u64 => AmmParams::LastOrderDistance,
        13u64 => AmmParams::InitOrderDepth,
        14u64 => AmmParams::SetSwitchTime,
        15u64 => AmmParams::ClearOpenTime,
        16u64 => AmmParams::Separate,
        17u64 => AmmParams::UpdateOpenOrder,
        _ => unreachable!(),
```

program/src/instructions.rs

```rust
impl SimulateParams {
  pub fn from_u64(flag: u64) -> Self {
    match flag {
        0u64 => SimulateParams::PoolInfo,
        1u64 => SimulateParams::SwapBaseInInfo,
        2u64 => SimulateParams::SwapBaseOutInfo,
        3u64 => SimulateParams::RunCrankInfo,
        _ => unreachable!(),
    }
```

BVSS

AO:A/AC:L/AX:L/R:P/S:U/C:N/A:L/I:N/D:N/Y:N (1.3)

Recommendation

Consider to refactor `AmmParams::from_u64` and `SimulateParams::from_u64` to return a `Result` type instead of invoking a panic, replacing the `_ => unreachable!()` branch with controlled error propagation via `Err(ProgramError::InvalidInstructionData)`, and handle the returned result using the `?` operator at the call site.

Remediation Comment

SOLVED: The `Raydium` team solved this issue by:

- Refactoring `AmmParams::from_u64` to return a `Result` and replacing the `_ => unreachable!()` branch with controlled error propagation
- Removing the `SimulateParams` / `SimulateInfo` deserialization path entirely.
- Replacing `.unwrap()` in `AmmParams::from_u64` from `AmmInstruction::unpack` (tag 6 – `SetParams`) with the `?` operator.

Remediation Hash

b20ba5685422a1a779583802ea691dce940f0475

9.3 UNCHECKED 32-BYTE READ IN UPDATECONFIGACCOUNT PANICS ON TRUNCATED INSTRUCTION DATA INSTEAD OF RETURNING A CONTROLLED ERROR

// INFORMATIONAL

Description

The deserialization logic for AMM instructions expects to decode instruction data into strongly typed variants that route subsequent processing. When handling UpdateConfigAccount (tag 15) with param set to 0 or 1, the unpacking flow must extract a 32-byte public key from the remaining instruction payload. The code uses the `array_ref!` macro from the `arrayref` crate to obtain a fixed-size slice reference directly from `rest`, assuming the buffer contains at least 32 bytes beyond the param field.

The issue arises because `AmmInstruction::unpack()` applies `array_ref![rest, 0, 32]` without first checking that `rest.len() >= 32`. The `array_ref!` macro internally performs slice indexing (`&rest[0..32]`), which panics at runtime if the slice is shorter than the requested length. As a result, a malformed instruction with a valid param byte (0 or 1) but fewer than 32 remaining bytes triggers an out-of-bounds panic instead of returning a controlled error.

Although this does not represent a security vulnerability, this behavior is inconsistent with the safe pattern already present in the same file: the analogous case for tag 6 (SetParams::AmmOwner) explicitly validates `rest.len() >= 32` before calling `array_ref!` and returns `ProgramError::InvalidInstructionData` when the buffer is too short. The absence of this check in tag 15 is therefore an omission rather than a deliberate design choice.

program/src/instructions.rs

Copy Code

```
599 | 15 => {
600 |   let (param, rest) = Self::unpack_u8(rest)?;
601 |   match param {
602 |       0 | 1 => {
603 |           let pubkey = array_ref![rest, 0, 32];
604 |           Self::UpdateConfigAccount(ConfigArgs {
605 |               param,
606 |               owner: Some(Pubkey::new_from_array(*pubkey)),
```

BVSS

AO:A/AC:L/AX:L/R:P/S:U/C:N/A:L/I:N/D:N/Y:N (1.3)

Recommendation

Consider to apply the same length guard used in tag 6 to the tag 15 deserialization path.

Remediation Comment

SOLVED: The `Raydium` team solved this issue by adding an explicit `rest.len() >= 32` length guard before the `array_ref![rest, 0, 32]` call in the tag 15 (`UpdateConfigAccount`) deserialization path, mirroring the safe pattern already used in tag 6.

Remediation Hash

02f351333e54b7fca573aad2f2b657ed6fe27903

9.4 STALE OPENBOOK ACCOUNTS IN BUILDERS MISALIGN THE SIGNER SLOT, BREAKING WITHDRAWPNL AND SET PARAMS

// INFORMATIONAL

Description

The migration that removed the OpenBook dependency updated the on-chain processors to stop consuming orderbook and market accounts, but the corresponding client-side instruction builders were not updated in parallel. The program resolves accounts positionally through sequential iteration, not by name.

As a consequence, accounts that a builder continues to include but the processor no longer reads positionally shift all subsequent accounts relative to what the processor expects. When this offset reaches a slot where the processor requires a signer, the account that falls into that position is not signed, and the runtime rejects the transaction.

Two instances have been verified:

- In `withdrawpnl`, the builder still passes the AMM open orders account that the processor skips, creating a one-position offset. The user's pc token account—not a signer—ends up in the slot where the processor expects the PNL owner signer, causing immediate rejection with an invalid sign account error.
- In `set_params`, the builder includes an eight-account OpenBook market block that the processor no longer reads, displacing the admin signer by eight positions. The processor reads the market program account where it expects the admin signer, and the signer check fails.

program/src/processor.rs

Copy Code

```
1284 pub fn process_withdrawpnl(program_id: &Pubkey, accounts: &[AccountInfo]) -> ProgramResult {
1285   let account_info_iter = &mut accounts.iter();
1286   let token_program_info = next_account_info(account_info_iter)?;
1287
1288   let amm_info = next_account_info(account_info_iter)?;
1289   let amm_config_info = next_account_info(account_info_iter)?;
1290   let amm_authority_info = next_account_info(account_info_iter)?;
1291   let amm_coin_vault_info = next_account_info(account_info_iter)?;
1292   let amm_pc_vault_info = next_account_info(account_info_iter)?;
1293   let user_pnl_coin_info = next_account_info(account_info_iter)?;
1294   let user_pnl_pc_info = next_account_info(account_info_iter)?;
1295   let pnl_owner_info = next_account_info(account_info_iter)?;
```

program/src/processor.rs

Copy Code

```
1317 if !pnl_owner_info.is_signer
1318     || (*pnl_owner_info.key != config_feature::amm_owner::ID
1319         && *pnl_owner_info.key != amm_config.pnl_owner)
1320 {
1321     return Err(AmmError::InvalidSignAccount.into());
1322 }
```

Although this does not represent an exploitable security vulnerability—the behavior is fail-closed, as the displaced account is not signed and the transaction is rejected before any state changes—the practical impact is that instructions generated by these builders are unusable. The scope is limited to client-side construction; on-chain logic is unaffected.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Consider to realign each builder with its processor by removing accounts the processor no longer consumes, ensuring positional order matches what the processor reads.

Remediation Comment

SOLVED: The `Raydium` team solved this issue by updating the client-side instruction builders to match the migrated processors as suggested. They removed the stale `OpenBook` accounts from the account lists so that every account once again lines up positionally with what the processor reads, placing the required signers back in the exact slots where the processor performs its `is_signer` checks.

Remediation Hash

02f351333e54b7fca573aad2f2b657ed6fe27903

9.5 MUTABLE AMM OWNER FIELD WITH NO EFFECT ON ACCESS CONTROL

// INFORMATIONAL

Description

The `AmmInfo` struct contains an `amm_owner: Pubkey` field that is initialized during pool creation to the compile-time constant `config_feature::amm_owner::ID` and can be modified by the admin via the `process_set_params` instruction with the `AmmParams::AmmOwner` parameter. The field's name and mutability suggest it defines the owner or administrator of the pool.

However, this stored field is never read in any access-control decision within the program. All privileged operations authorize against the hardcoded constant `config_feature::amm_owner::ID` (or `config_feature::amm_owner::id()`), not against the value stored in the pool account. A complete code review confirms that the only references to `amm.amm_owner` are two writes (during initialization and parameter updates) and one read exclusively in tests; no execution logic consumes this field.

The issue arises because the program presents a misleading API surface. An operator invoking `AmmParams::AmmOwner` expects to transfer or delegate administrative control of a specific pool, but the stored value has zero effect on program behavior.

Authorization remains securely anchored to the compile-time constant across all flows—`process_set_params`, `process_withdrawpnl`, `process_create_config`, and `process_update_config` all check the caller's key directly against `config_feature::amm_owner::ID` or the config account's `pnl_owner` field, never against `amm.amm_owner`. The field occupies 32 bytes of pool account space that the admin can modify arbitrarily without any functional consequence, unnecessarily expanding the configuration surface.

Although this does not represent an exploitable vulnerability—there is no privilege escalation, authorization bypass, or loss of funds, and authorization remains safely anchored to the compile-time constant in all flows—it is recommended to resolve the inconsistency between the field's name, mutability, and actual utility.

program/src/processor.rs

Copy Code

```
884 | amm.amm_owner = config_feature::amm_owner::ID;
```

Copy Code

```
2746 | AmmParams::AmmOwner => {
2747 |   let new_pubkey = match setparams.new_pubkey {
2748 |       Some(a) => a,
2749 |       None => return Err(AmmError::InvalidInput.into()),
2750 |   };
2751 |   amm.amm_owner = new_pubkey;
```

program/src/processor.rs

```
2629 if !amm_owner_info.is_signer || *amm_owner_info.key != config_feature::amm_owner::ID {
2630     return Err(AmmError::InvalidSignAccount.into());
2631 }
```

```
1317 if !pnl_owner_info.is_signer
1318       || (*pnl_owner_info.key != config_feature::amm_owner::ID
1319           && *pnl_owner_info.key != amm_config.pnl_owner)
1320   {
1321       return Err(AmmError::InvalidSignAccount.into());
1322   }
```

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Resolve the inconsistency by choosing one of two approaches:

- **Remove the misleading surface**: Remove the `AmmParams::AmmOwner` branch from `process_set_params`, leaving the field as read-only informational (or removing it entirely if account layout compatibility permits), so that no operator assumes a delegation capability that does not exist.
- **Give the field effect**: Modify relevant authorization checks to read `amm.amm_owner` in addition to—or instead of—the hardcoded constant, enabling real per-pool administrative delegation. This option introduces new privilege logic and should be carefully audited to prevent misconfiguration from locking out legitimate operations.

Remediation Comment

SOLVED: The `Raydium` team solved this issue by removing the `AmmParams::AmmOwner` variant from the `AmmParams` enum, eliminating the only instruction path that allowed the admin to mutate the `amm_owner` field via `process_set_params`, so the misleading mutable API surface no longer exists.

Remediation Hash

02f351333e54b7fca573aad2f2b657ed6fe27903

9.6 RESIDUAL OPENBOOK VALIDATION WITH NO EFFECT DURING SETTING PARAMS

// INFORMATIONAL

Description

After the OpenBook dependency was removed, `process_set_params` still retains a legacy validation that checks the passed `amm_open_orders_info` account against the stored `amm.open_orders` field:

program/src/processor.rs

```
2607 | let amm_open_orders_info = next_account_info(account_info_iter)?;
```

program/src/processor.rs

```
2632 | if *amm_open_orders_info.key != amm.open_orders {
2633 |     return Err(AmmError::InvalidOpenOrders.into());
2634 | }
```

This check no longer enforces any meaningful boundary:

- For pools initialized after the OpenBook removal, `amm.open_orders` is hardcoded to `Pubkey::default()` (the all-zero / System Program address) at init and is never modified afterward. For these pools the check is satisfied by passing the System Program account.
- For legacy pools (created before the removal), the field retains its original OpenBook open-orders pubkey, loaded from stored state via `AmmInfo::load_mut_checked`.

program/src/processor.rs

```
880 | amm.pc_vault = *amm_pc_vault_info.key;
881 | amm.coin_vault_mint = *amm_coin_mint_info.key;
882 | amm.pc_vault_mint = *amm_pc_mint_info.key;
883 | amm.lp_mint = *amm_lp_mint_info.key;
884 | amm.open_orders = Pubkey::default();
```

program/src/processor.rs

```
2629 | if !amm_owner_info.is_signer || *amm_owner_info.key != config_feature::amm_owner::ID {
2630 |     return Err(AmmError::InvalidSignAccount.into());
2631 | }
```

Although this does not represent a security vulnerability—`process_set_params` remains protected by the admin-signer check that verifies both signature presence and that the signer matches the hardcoded admin key in `config_feature::amm_owner::ID`, and the trivial open-orders validation grants no additional capability to unauthorized actors—it is recommended to remove this residual code.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to Remove the residual validation and discard the account binding.

Consider removing the `open_orders` field from `AmmInfo` and the `AmmError::InvalidOpenOrders` variant if OpenBook is fully phased out, after assessing compatibility impact.

Remediation Comment

SOLVED: The `Raydium` team solved this issue by removing the residual open-orders validation entirely from `process_set_params`, along with the now-unused `amm_open_orders_info` account. The instruction now deserializes only `amm_info` and `amm_owner_info`, relying solely on the admin-signer check against `config_feature::amm_owner::ID` for authorization.

Remediation Hash

02f351333e54b7fca573aad2f2b657ed6fe27903

9.7 STATISTICAL FIELDS IN STATEDATA REMAIN PERMANENTLY AT ZERO

// INFORMATIONAL

Description

The `StateData` struct contains on-chain telemetry fields designed to track pool activity metrics: `total_pnl_pc`, `total_pnl_coin`, `swap_coin_in_amount`, `swap_pc_out_amount`, `swap_pc_in_amount`, `swap_coin_out_amount` (volume counters), and `swap_acc_pc_fee`, `swap_acc_coin_fee`.

These fields are initialized to zero in `StateData::initialize` and are expected to accumulate statistics throughout the pool's lifecycle to provide on-chain visibility into trading volume, profit-and-loss, and fee collection.

The issue occurs because the accumulation logic that previously updated these fields was removed. The update calls were stripped from both `take_pnl` and all four swap instruction handlers.. As a result, the fields remain at their initial zero values indefinitely, even as swaps execute and fees accumulate in the vault.

The fee collection mechanism itself continues to function correctly—`amount_in` is deposited in full and only `swap_in_after_deduct_fee` is used to compute output)—but the on-chain accounting fields are never incremented to reflect actual activity.

program/src/state.rs

Copy Code

```
615 pub fn initialize(&mut self, open_time: u64) -> Result<(), AmmError> {
616   self.need_take_pnl_coin = 0u64;
617   self.need_take_pnl_pc = 0u64;
618   self.total_pnl_pc = 0u64;
619   self.total_pnl_coin = 0u64;
620   self.pool_open_time = open_time;
621   self.padding = Zeroable::zeroed();
622   self.orderbook_to_init_time = 0u64;
623   self.swap_coin_in_amount = 0u128;
624   self.swap_pc_out_amount = 0u128;
625   self.swap_acc_pc_fee = 0u64;
626   self.swap_pc_in_amount = 0u128;
627   self.swap_coin_out_amount = 0u128;
628   self.swap_acc_coin_fee = 0u64;
```

program/src/processor.rs

Copy Code

```
1889 let swap_fee = U128::from(swap.amount_in)
1890     .checked_mul(amm.fees.swap_fee_numerator.into())
1891     .unwrap()
1892     .checked_ceil_div(amm.fees.swap_fee_denominator.into())
1893     .unwrap();
1894 let swap_in_after_deduct_fee = U128::from(swap.amount_in).checked_sub(swap_fee).unwrap();
1895 let swap_amount_out = Calculator::swap_token_amount_base_in(
1896     swap_in_after_deduct_fee,
1897
```

```
        total_pc_without_take_pnl.into(),
        total_coin_without_take_pnl.into(),
        swap_direction,
    )
    .as_u64();
```

Copy Code

```
amm.state_data.need_take_pnl_coin = 0u64;
amm.state_data.need_take_pnl_pc = 0u64;
// update target_orders.calc_pnl_x & target_orders.calc_pnl_y
target_orders.calc_pnl_x = x1.checked_sub(U128::from(delta_x)).unwrap().as_u128();
```

The impact is strictly informational and does not affect fund safety or swap correctness. The pool's economic invariants and fee retention remain intact. However, any off-chain consumer (SDKs, frontends, indexers, or third-party integrators) that deserializes StateData directly from the account will read zeros instead of historical cumulative values. Since these fields are not emitted in any event logs, direct deserialization is the only consumption path. While the Raydium team confirmed they do not rely on these fields internally, they cannot guarantee that no external integrator depends on them for analytics or reporting.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Consider to explicitly document that these `StateData` fields are deprecated and will remain at zero, given that fee, volume, and PnL metrics can be reconstructed off-chain from swap event logs (`SwapBaseInLog` / `SwapBaseOutLog`) and transaction history. Also, communicate this in SDK and integration release notes to prevent external consumers from interpreting the zero values as actual data.

Remediation Comment

SOLVED: The `Raydium` team solved this issue by removing the dead accumulation calls from `take_pnl` and the four swap handlers, and explicitly documenting the affected `StateData` fields as deprecated fields that will remain at zero, while preserving them in the struct to keep the on-chain account layout backward-compatible.

Remediation Hash

02f351333e54b7fca573aad2f2b657ed6fe27903

9.8 INCONSISTENT COMPARISON IN PARAMS UPDATE PREVENTS REDUCING MAX PRICE MULTIPLIER

// INFORMATIONAL

Description

The `process_set_params` function handles updates to AMM configuration parameters through a match statement on `AmmParams` variants. The `AmmParams::MaxPriceMultiplier` case is expected to validate the new value against min_price_multiplier to preserve the invariant min < max, mirroring the validation logic in the `AmmParams::MinPriceMultiplier` case, which correctly checks value < `amm.max_price_multiplier` before updating. This symmetrical validation ensures that the admin can adjust both bounds of the price multiplier range while maintaining their relative ordering.

The issue arises because the `MaxPriceMultiplier` case validates the incoming value against the field it intends to modify—amm.max_price_multiplier itself—rather than against the opposing bound

This condition is only satisfied when value strictly exceeds the current `max_price_multiplier`, effectively creating a unidirectional ratchet: the field can only increase, never decrease. The expected check, consistent with the MinPriceMultiplier validation, is `value > amm.min_price_multiplier`.

Although this does not represent a security vulnerability—the set_params instruction is restricted to the `config_feature::amm_owner::ID signer`, and `min_price_multiplier` and `max_price_multiplier` are currently inactive fields not consumed by any swap, pricing, or calculation logic—the inconsistency prevents the admin from lowering the upper bound once set. If these fields are reintroduced into active logic in future versions, the unidirectional ratchet would become an operational constraint. It is recommended to align the validation logic with the `MinPriceMultiplier` case for consistency and to prevent future regression.

program/src/provcessor.rs

Copy Code

```
2697 AmmParams::MinPriceMultiplier => {
2698   let value = match setparams.value {
2699       Some(a) => a,
2700       None => return Err(AmmError::InvalidInput.into()),
2701   };
2702   if value < amm.max_price_multiplier {
2703       amm.min_price_multiplier = value;
2704       set_valid = true;
2705   }
```

Copy Code

```
2707 AmmParams::MaxPriceMultiplier => {
2708   let value = match setparams.value {
2709       Some(a) => a,
2710       None => return Err(AmmError::InvalidInput.into()),
2711   };
2712   if value > amm.max_price_multiplier {
2713       amm.max_price_multiplier = value;
2714
```

```
2715            set_valid = true;
        }
```

Copy Code

```
2629 if !amm_owner_info.is_signer || *amm_owner_info.key != config_feature::amm_owner::ID {
2630     return Err(AmmError::InvalidSignAccount.into());
2631 }
```

Copy Code

```
792 self.min_price_multiplier = 1;
793 self.max_price_multiplier = 1000000000;
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Consider to align the comparison with the MinPriceMultiplier case to allow bidirectional updates.

Remediation Comment

SOLVED: The `Raydium` solved this issue by removing the `MinPriceMultiplier` and `MaxPriceMultiplier` variants from the `AmmParams` enum and their corresponding arms in `process_set_params`, eliminating the code path that contained the inverted validation so the unidirectional ratchet is no longer reachable.

Remediation Hash

02f351333e54b7fca573aad2f2b657ed6fe27903

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.